

# Event-Driven Hierarchical Rule Composition and Closure Sharing

From Execution Histories to Dissipative and Infinite Fractal Rules

Technical report 21 September 2026

Implementation baseline: 258eef4

## Abstract

Equality saturation shares concrete expressions, but repeated derivations with different bindings can still generate large graphs and dense execution histories. We present a hierarchy that treats rule applications as evidence for reusable reasoning units. Committed events form a versioned dependency graph. Relative to a selected context, applications are classified as coarse or smooth and organized into coarse layers, smooth layers, and CS units. Sequential CSCS composition preserves dependencies; guarded CCSS regrouping combines the coarse and smooth parts without discarding interface obligations. An explicit *ripen* operation computes local closure under staged external inputs, while exact state comparison preserves types, equality, visibility, and the relevant rule environment. We then define parameterized fractal rules through stable binding and effect transfers, distinguishing well-founded dissipative processes from families with arbitrarily long finite prefixes. These abstractions suggest finite scheduling, endpoint reduction, and demand-driven symbolic feedback to ripen. A native egglog prototype implements bounded composition, staged validation, closure sharing, event subscriptions, and recursive promotion. A three-unit example yields nine closed results, seven shared states, and six promotions, reaching depth two. This establishes a working finite mechanism, not end-to-end tier0 compression or acceleration. General induction and complete fractal-summary feedback remain explicitly separated from the implemented capabilities.

**Keywords:** equality saturation; rule composition; local closure; relative binding; event-driven analysis; recursive summaries.

## 1 Introduction

An e-graph shares e-nodes with identical operators and canonical children, and records equivalence between expressions through union. This eliminates concrete structural duplication. It does not automatically share a derivation such as expansion, commutation, reassociation, and folding across many different bindings. Even when the expression graph shares some nodes, the history of matches and the space of potential rule combinations may continue to grow.

Our objective is to discover reusable reasoning units that can eventually reduce repeated storage and matching. Three objects must remain distinct: a state, a process reaching that state, and the conditions enabling the process. Different processes may reach the same state, while the same rule sequence may behave differently under different bindings and external effects. Neither a list of rule names nor a single endpoint expression is therefore an adequate sharing key.

Egglog combines Datalog-style reasoning with equality saturation and provides the native execution substrate for our prototype [1]. We use *tier0* for ordinary execution, *tier1* for the analysis of applications and their compositions, and *tier2* for binding transfers, recurrence evidence, and symbolic reduction. These are logical layers: the current tier1 is primarily a Rust data structure rather than a mandatory chain-shaped egglog encoding.

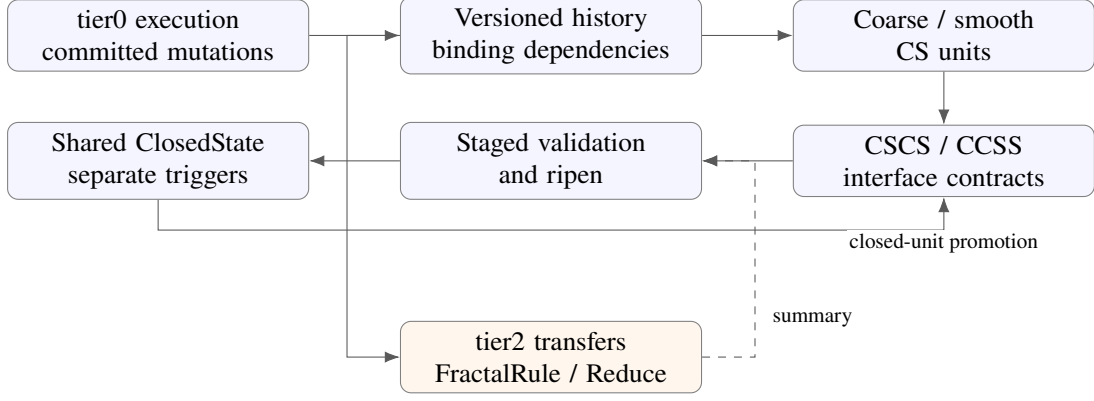


Figure 1: The hierarchy. The dashed connection indicates certified-summary feedback that is not yet automatically integrated into ripen. Finite closure promotion is implemented.

The contribution of this report is a semantic account of this hierarchy, together with a bounded implementation and explicit evidence limits. We do not claim a complete enumeration of useful combinations, a general induction procedure, or proven runtime compression. The central design decision is to use observed interaction events to construct larger units, and to share their closed contents without conflating their triggers.

## 2 State and Execution Evidence

### 2.1 Facts, equality, visibility, and updates

We distinguish the components of a runtime state as

$$G = (F, \equiv, V, \Theta, \delta). \quad (1)$$

Here  $F$  contains constructor and relation facts,  $\equiv$  is equality,  $V$  records row visibility,  $\Theta$  contains function tables and their update semantics, and  $\delta$  denotes execution and incremental-maintenance state. The supplied rewrite theory is trusted; native replay is not an independent proof of every rule as a mathematical identity. A union is not an ordinary row insertion; subsumption is not simply the absence of a semantic fact; and a function update is not necessarily set union.

For a positive fragment that only adds facts and grows equality, a monotone closure model is useful. General programs require a stronger operational contract. A subsumed row can remain stored while ceasing to participate in matching. Comparing only its contents would lose this distinction.

### 2.2 Committed applications rather than matches

An application record is modeled as

$$a_i = (R_i, \beta_i, Q_i, \Delta_i, U_i, W_i), \quad (2)$$

where  $R_i$  is a rule,  $\beta_i$  its binding,  $Q_i$  the facts and equality evidence read,  $\Delta_i$  the committed writes,  $U_i$  the union information, and  $W_i$  update or visibility information. A logical match, a surviving action lane, an inserted row, and a deduplicated write are different events. Our provenance must not silently identify them.

An edge from  $a_i$  to  $a_j$  records that  $a_j$  used a then-valid fact version or equality witness supplied by  $a_i$ . Equal values can suggest interaction, but value equality alone does not establish this causal edge. The event graph is acyclic in time even when a rule consumes its own earlier output. Cycles in a quotient graph of recurring patterns instead denote recursion between pattern instances, not cycles in the original event history.

### 2.3 Relative binding as an interface map

A relative binding maps an input of one unit to an output, a carried input, or a new external port:

$$\rho : \text{In}(B) \longrightarrow \text{Out}(A) \cup \text{In}(A) \cup E. \quad (3)$$

It also carries types, alias constraints, and equality evidence. Such a map may be a permutation, a projection through constructor arguments, a construction of new terms, or a reference to another application's result. The notation  $\beta' = \beta + \Delta\beta$  is appropriate only for an actually additive coordinate system; it is not a general definition of relative binding.

Definitions and uses follow a dependency order. A later application may refer to an earlier binding, but a forward reference or a collision between names from different instances is invalid. Numbering variables by first appearance alone is not a complete canonicalization of arbitrary branching graphs.

## 3 CoarseComb and SmoothComb

Let  $H$  be a selected local unit. Immediately before its  $i$ th stage, let  $A_i(H)$  be the evidence supplied by already injected inputs and earlier internal actions. For the chosen recorded witness, define the new external obligation conceptually as

$$E_i(H) = Q_i \setminus A_i(H). \quad (4)$$

The new injection for stage  $i$  is not yet included in  $A_i(H)$ . By convention, the initial triggering input is charged to the first coarse step. This is evidence subtraction, not subtraction of bare IDs: equality, versions, and visibility must be respected.

**Definition 1** (Context-relative application kind). *An application is a CoarseComb relative to  $H$  if it requires a nonempty new external obligation  $E_i(H)$ . It is a SmoothComb if all its premises are explained by the current local interface and prior internal results.*

The classification belongs to an application in a chosen combination, not to the rule permanently. A coarse application can become smooth when its suppliers are included in a larger unit. Classification also depends on the selected witness: a different internal proof may exist without having been discovered. Missing provenance is not proof that no such internal derivation exists.

For example, take  $R_1 : A(x) \Rightarrow B(x)$  and  $R_2 : B(x) \Rightarrow C(x)$ . Starting from an external  $A(v)$ , the first application is coarse and the second is smooth in the two-step combination. Viewed in isolation, the second application is coarse because  $B(v)$  is external. If it additionally requires  $P(v)$ , that condition cannot be discarded merely because most inputs come from the first rule.

A nonempty-premise combination normally begins with a coarse injection. A rule with no premises is a logical exception. An engineering convention such as `Empty -> CoarseComb` can provide a uniform entry representation, but does not prove that every rule needs external data.

## 4 Layers and CS Units

A *coarse layer* groups coarse applications with explicit binding anchors. A *smooth layer* contains applications whose relevant dependencies can be explained behind that interface. We write a CS unit as

$$U = (C, S, I, O, \Pi), \quad (5)$$

with coarse members  $C$ , smooth members  $S$ , external interface  $I$ , output ports  $O$ , and witness information  $\Pi$ .

An intersection is a reason to inspect a pair, not automatically a reason to merge it. For example, merging transitively on the shared constant zero can destroy locality. A common smooth consumer,

explicit port connections, candidate budgets, and coverage checks provide stronger selection signals. Requiring a consumer to use outputs from each participating coarse member is a useful heuristic, not a semantic-equivalence theorem.

Units need not form a disjoint partition. A fact can be exposed by several interfaces, and a history can admit multiple useful decompositions. Sharing references and retaining alternatives avoids permanently committing to an early partition.

A *Use* is a concrete occurrence of an installed rule-combination template. It records a template ID, member events, and actual port bindings. This differs from a CS unit, which is defined by contextual obligations. The earlier pipeline admitted only Uses to ripen. The current pipeline can create CS candidates even when no template has been installed; Use remains an alternative candidate source and an experimental baseline.

## 5 Two Composition Operators

### 5.1 Sequential CSCS composition

If  $U_2$  consumes outputs of  $U_1$ , we represent their composition as

$$\text{Seq}(U_1, U_2, \rho). \quad (6)$$

The map  $\rho$  separates inputs supplied by  $U_1$  from inputs that still require external injection. It preserves the dependency direction rather than simply concatenating two event arrays. If both units contain applications depending on intermediate results of the other, the complete units cannot be placed in a simple sequential order. A finer decomposition or an unknown result is needed. Dependencies through events outside the proposed pair must also be checked.

### 5.2 Guarded CCSS regrouping

The second operator regroupes the coarse and smooth parts:

$$\text{Par}(U_1, U_2, \alpha) = (C_1 \cup C_2, S_1 \cup S_2, I', O', \Pi'). \quad (7)$$

Here  $\alpha$  gives typed anchors between the C parts. The name does not promise parallel execution. Dependencies within the combined C or S DAG remain edges.

Our sufficient admission conditions are conservative: disjoint member events, an explicit coarse-port intersection, supported positive constructor/relation/equality operations, and no transitive dependency from any S member to any C member. An exhausted bounded search is unknown, not evidence of independence.

**Proposition 1** (Conditional regrouping). *Fix the external-input obligations and a complete dependency witness. If no C member depends on an S member, a topological order exists that places C before S. For supported positive monotone actions, changing between legal internal orders preserves the resulting deductive closure.*

*Proof.* All predecessors of C lie outside S, so the C subgraph can be topologically ordered before the S subgraph. Every selected premise remains available because internal producers precede consumers and the external obligations are fixed. Independent additions and unions do not retract earlier evidence. Recomputing closure under the same positive theory therefore preserves the consequences.  $\square$

This proposition does not move a late external fact earlier. The implementation validates the original staged sequence; its CCSS certificate is conditional on interface obligations, not a claim that the surrounding tier0 execution has been rescheduled. Function updates, subsume, and unsupported guards do not pass the positive-fragment regrouping check.

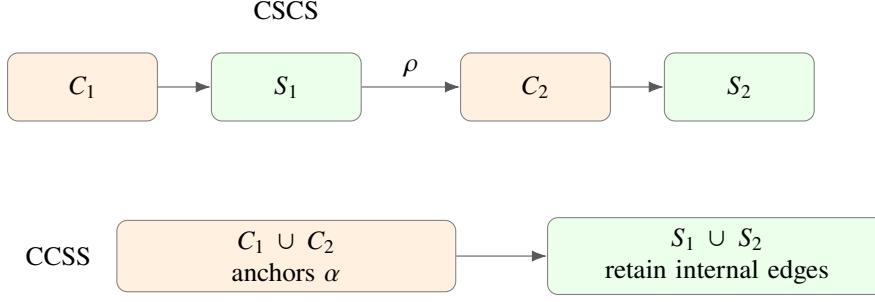


Figure 2: CSCS preserves a cross-unit dependency. CCSS regroupes C and S only under its contract. Neither operation removes external-input obligations.

### 5.3 Anchors need not be an initial intersection

Suppose one unit supplies  $P(a, b)$ , another supplies  $Q(c, d)$ , and a recorded consumer also reads  $Link(b, c)$ . The consumer and the Link fact anchor a combination even if the original C parts shared no node. This is a more general interaction witness than a direct coarse intersection. It must not be forced into the particular CCSS admission test.

The prototype stores explicit composition references. Closed compositions can be promoted and become operands of later compositions. Their inherited coarse and smooth descriptions are still conservative: promotion does not automatically find the best reclassification or all legal regroupings.

## 6 Ripen and Closure Sharing

### 6.1 Local closure under staged inputs

Given a rule environment  $\mathcal{R}$ , a local entry  $I$ , and an injection plan  $J$ , ripen replays the observed preconditions, injections, and actions, then runs the local rules until a complete sweep makes no update or a budget is reached. The statuses are deliberately distinct:

- Pending: not yet processed;
- Suspended: insufficient execution budget;
- Rejected: entry or validation obligations were not met;
- Closed: an operational fixed point under the specified local model.

A local Closed result neither saturates all of tier0 nor certifies every embedding of the interface.  $RipenInput(k)$  is an opaque symbolic boundary node: a repeated index preserves identity, while different indices are initially unmerged. It is not a wildcard that matches every value. It does not recover the internal structure of an external eclass. Matches requiring that structure must be treated as additional coarse interaction.

If a second C requires a later external fact, the contract has the form

$$I_1; C_1; S_1; Inject(I_2); C_2; S_2. \quad (8)$$

Injection declares a caller obligation; it does not derive the fact internally. Promoting  $I_2$  into the initial entry without justification would change the activation conditions.

### 6.2 Sharing contents without conflating triggers

Different combinations may share identical closed contents while retaining different triggers, bindings, and injection plans. The prototype compares complete exported typed graphs, equality classes, fixed ports, and

row visibility. The rule environment must match. When updates or visibility matter, registration order and ruleset scheduling information are also retained.

**Proposition 2** (Composition of monotone closures). *If  $\text{Cl}$  is extensive, monotone, and idempotent over a fixed universe, then*

$$\text{Cl}(\text{Cl}(A) \cup \text{Cl}(B)) = \text{Cl}(A \cup B). \quad (9)$$

*Proof.* Extensiveness gives  $A \cup B \subseteq \text{Cl}(A) \cup \text{Cl}(B)$ , yielding one inclusion by monotonicity. Conversely, both  $\text{Cl}(A)$  and  $\text{Cl}(B)$  are contained in  $\text{Cl}(A \cup B)$ . Monotonicity and idempotence give the reverse inclusion.  $\square$

This identity motivates starting from existing closures and processing new interaction. Their union need not already be closed. The identity supplies no execution budget and is not a theorem about every egglog operation. In particular, subsume changes visibility, and a min/max merge alone does not prove that every surrounding query is monotone.

Exported-content equality is also not equivalence of arbitrary suspended incremental engines: the key does not serialize all of  $\delta$ . Direct reuse of native matcher or join state is not implemented. The current guarantee is sharing of declared local contents with separate trigger contracts.

### 6.3 Versioned evidence and recursive promotion

Historical closed contents are immutable; the applicability of a concrete instance is separately versioned. Token, table, and scope subscriptions are notified by committed changes. Union joins subscriptions so later notifications follow aliases. Updates and subsumption expire affected evidence. Unknown row ownership causes a conservative table- or scope-level invalidation.

A stale descriptor can propose another symbolic candidate, but cannot bypass native staged validation. Certificates retain `live_substitution=false`: a lease whose version has not changed is not a proof of a current whole-graph embedding. Successful closed compositions are promoted as new descriptors and registered for further pairing, without replacing tier0 facts.

## 7 Parameterized Fractal Rules

### 7.1 Stable binding and effect transfers

Repeated rule names are weak evidence. Two ABAB sequences can have different bindings; conversely, independently scheduled branches can represent the same dependency structure despite different event orders. We model a repeated step as

$$z_{k+1} = T(z_k), \quad z_k = (\beta_k, \mathcal{E}_k, \mathcal{P}_k), \quad (10)$$

where  $\beta_k$  is the binding,  $\mathcal{E}_k$  accumulated effects, and  $\mathcal{P}_k$  the frontier of positions and obligations. In an operational fragment, the effect component must retain structured updates and visibility, not merely a set of inserted facts.

A `FractalRule` is a parameterized family with a trigger, guard, stable transfer, effect contract, and observable interface. “Fractal” denotes recursive structural regularity here, not a claimed geometric dimension. A useful abstract form is

$$\mathcal{F} = (\text{Trigger}, D, T, \text{Guard}, \text{Effects}, \text{Queries}). \quad (11)$$

Finite histories propose such an object; they do not prove that it remains valid for arbitrary counts. The current transfer analysis records Carry, Return, and External port origins and finite witnesses. General induction remains a separate proof obligation.

## 7.2 Trigger states and nonuniform prefixes

A stable phase may begin only after several injections and effect-producing steps:

$$\text{Trigger}(I, \beta_0); T^k; \text{Exit}. \quad (12)$$

Different bindings may require different triggers. An initial “dirty” prefix must not be omitted merely because the later transfer is regular.

For example, swapping Add arguments twice is an identity on argument positions, but the first swap may insert a node and perform a union. For a fixed position and fixed arguments, once both orders and their equivalence are present, further swaps add no local effect. The stabilization point depends on the initial graph; there is no universal rule equating two swaps with an empty execution.

Nor does a change of rule name in the log establish a turning point. Semi-naive scheduling and other work can interrupt a still-enabled family. A turning point must be defined by a guard, resource, interface, or effect change. Fractal operators need not commute or be orthogonal.

## 7.3 Multiple application positions

For a branching family, use an indexed position domain rather than flattening all steps into a sequence. A schematic binary generator can use

$$D_{k+1} = \{2i, 2i + 1 \mid i \in D_k\}. \quad (13)$$

The text  $A \rightarrow B + B$  alone does not prove physical doubling: identical B nodes can be hashconsed. Exponential e-graph growth additionally requires distinct bindings or positions that are not absorbed by existing equality. Three-way branching generally suggests  $3^k$ , not cubic growth in  $k$ .

We must therefore distinguish unfolded tree size, parameterized description size, event count, and deduplicated e-node count. A compact binding generator can represent a large family even when the resulting graph does not compress by the same factor.

# 8 Dissipative and Infinite-Prefix Families

## 8.1 Dissipation is a property of obligations

**Definition 2** (Dissipative FractalRule). *Under a fixed external-supply contract, a fractal process is dissipative if a well-founded measure strictly decreases on every valid recursive step. A simple case is  $\mu(z_k) \in \mathbb{N}$  with  $\mu(z_{k+1}) < \mu(z_k)$ .*

The measure applies to progress steps of the represented process; redundant re-execution of an already accounted match is not a new recursive step. The decreasing quantity may count remaining positions, an unprocessed index range, or outstanding obligations. It does not necessarily count facts being deleted from the e-graph. Facts are generally reusable, so a need for external facts does not itself imply dissipation. Replenishing the external supply may invalidate a phase-local ranking argument.

For a finite sum,

$$(i, s) \mapsto (i + 1, s + x_i), \quad 0 \leq i < n, \quad (14)$$

$\mu = n - i$  proves termination. In reducing five ones to five, it is the remaining reduction positions that decrease. The constant node one can remain a single shared node throughout, and need not disappear at all.

A branching process can also be dissipative while its levels become wider. Suppose each task of rank  $r > 0$  creates at most  $d$  tasks of rank at most  $r - 1$ , and rank-zero tasks terminate. On a finite frontier, use

$$\mu = \sum_{p \in \mathcal{P}} (d + 1)^{r(p)}. \quad (15)$$

Replacing a positive-rank task adds at most  $d(d+1)^{r-1}$ , less than the removed weight  $(d+1)^r$ . Thus bounded-depth binary expansion can have exponentially wide levels and still terminate. The removed objects in this argument are pending tasks, not retained e-graph facts.

Repeated integration by parts offers another candidate: polynomial degree may decrease if the correct factor and variable are repeatedly differentiated. This needs a polynomial and differentiation contract; finite examples do not prove that arbitrary integral rewrites terminate. We do not report such a general proof as an implemented capability.

## 8.2 InfiFractalRule as an unbounded prefix interface

**Definition 3** (Infinite-prefix family). *For a fixed admitted trigger instance, an InfiFractalRule admits, under its invariant, a valid prefix of length  $k$  for every natural number  $k$ . Claiming unbounded growth of the physical graph further requires a freshness argument for facts or positions.*

The initial instance is fixed: choosing a larger terminating input separately for each  $k$  does not establish this property. This definition concerns a family of finite prefixes. With nondeterministic choices, arbitrarily long paths do not by themselves establish that one selected execution continues forever. A fixed transfer and consistent prefixes provide a stronger contract.

We distinguish stabilization of effects, bounded periodic representations, and genuinely unbounded generation. Pure smooth execution does not imply the last case, and budget exhaustion proves none of them. Unknown remains an essential outcome.

For example, mathematical-natural-number payloads can describe  $A(k, i) \rightarrow B(k, 2i)$ ,  $B(k, 2i+1)$  and  $B(k, i) \rightarrow A(k+1, i)$ . Machine i64 arithmetic additionally requires overflow guards or bounded depth; it cannot inherit an unbounded mathematical conclusion without qualification.

## 9 Feedback from FractalRule to Ripen

### 9.1 Finite scheduling and reduction

A proved dissipative family supplies a principled local count or budget. Ripen can execute its trigger, finite repeat, and exit as a unit, rechecking the contract when external supply or binding changes. A ranking for one family does not prove that all of ripen terminates: another family may reactivate it.

The existing tier2 DSL includes an explicit Reduce node and exact endpoint laws. For a natural count  $k$  and a fixed per-step contribution  $c$ ,

$$\text{Reduce}(\text{AddConstant}(c), k, t) = t + kc. \quad (16)$$

For contributions  $a + id$ ,  $i = 0, \dots, k-1$ , the accumulated contribution is

$$ka + \frac{k(k-1)}{2}d. \quad (17)$$

These are laws for a supplied accumulation contract, not an algorithm discovering that contract from arbitrary history. They concern exact algebra; floating-point reassociation and machine overflow require additional justification.

### 9.2 A worked endpoint and intermediate-fact example

Consider the guarded recurrence

$$R : f(t) \Rightarrow f(t-1) + 1, \quad t > 0. \quad (18)$$

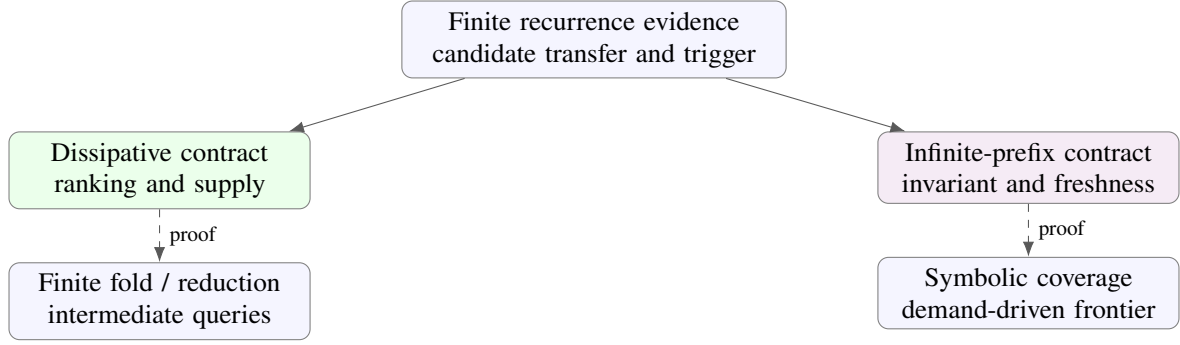


Figure 3: Proof obligations separate observed regularity from summary-based execution. A finite witness is not a certificate of either termination or unbounded generation.

For natural  $n$  and  $0 \leq k \leq n$ , induction yields the endpoint  $f(n - k) + k$ . Setting  $k = n - m$  gives

$$f(n) \Rightarrow f(m) + n - m, \quad 0 \leq m \leq n. \quad (19)$$

The guard provides a dissipative measure. Removing it over mathematical integers instead permits an unbounded descending sequence, subject to a freshness and effect argument; the same machine-integer implementation is still bounded by its numeric semantics.

The endpoint does not contain every intermediate fact. Another rule may need  $f(n - j)$  at a particular  $j$ . A reusable summary therefore needs a query interface for  $0 \leq j \leq k$ , together with the corresponding equality/effect information. Replacing the entire history with only the last expression can suppress such interactions and is not a sound implementation of general ripen.

### 9.3 Arrays and symbolic coverage

A useful summary contract exposes a trigger, iteration domain, binding transfer, effects, endpoint queries, and intermediate-position queries. Finite domains naturally give fold and array abstractions; infinite-prefix families require lazy instantiation. Existing logical ArrayGet, Tabulate, Map, and Fold constructs provide relevant vocabulary, but do not by themselves implement mutable memory, alias analysis, cache placement, or GPU scheduling.

An InfiFractalRule that continually generates fresh facts generally has no finite ordinary ClosedState. We propose a distinct SymbolicallyCovered status: within a specified observation interface, internal matches and effects are represented by a summary and required positions can be produced on demand. This needs both sound answers and coverage of relevant internal matches. If a query cannot be answered, ripen must retain a frontier, expand the requested position, or return unknown. It must not label budget-limited sampling as closure.

### 9.4 Bake and the present integration boundary

Bake can aggregate observed families from multiple examples into a frozen library. A new example matches the library under explicit binding/effect contracts rather than silently assuming that training examples proved generality. The local implementation already includes narrow, name-independent proof schemas for a guarded unit-stride scalar recurrence and a bounded radix-frontier payload query [2]. The former preserves the limit and other fields; the latter checks residue coverage and machine-range bounds. Their certified queries do not materialize all tier0 effects.

These narrow queries are useful evidence that executable summaries are possible. They are not a complete FractalRule-to-ripen integration. The current native ripen path still reports zero fractal summaries used. General induction and a complete internal-match interface remain open.

| Component            | Implemented scope                                                           |
|----------------------|-----------------------------------------------------------------------------|
| Native history       | Committed applications, binding evidence, mutation records, replay.         |
| Layers and Uses      | Context-relative coarse/smooth structures and template instances.           |
| CS composition       | Bounded references, conditional certificates, staged validation, promotion. |
| Closed-state sharing | Bounded exact comparison, typed ports, row visibility, rule environment.    |
| Evidence leases      | Token/table/scope subscriptions, union aliases, invalidation.               |
| Recurrence analysis  | Finite transfer witnesses and selected symbolic endpoint laws.              |
| Bake queries         | Narrow checked scalar summaries; no whole-program mutation replacement.     |
| General feedback     | Induction and complete symbolic match coverage are not integrated.          |

Table 1: Implementation scope, based on local source and tests [2, 3].

Listing 1: Target feedback protocol. The certified-summary branches are not yet generally integrated.

```

on committed event e:
  notify subscribers of changed resources
  update witnessed CS interfaces
  propose bounded CSCI and guarded CCS candidates

for candidate U within budget:
  validate staged external obligations and bindings
  if a verified dissipative summary applies:
    guide ripen using its finite domain and effect contract
  else if a verified infinite summary covers required queries:
    retain SymbolicallyCovered and a demand-driven frontier
  else:
    run ordinary bounded ripen
  share closed contents, retaining trigger obligations
  promote certified results and notify subscribers

```

## 10 Implementation and Evidence

### 10.1 What is implemented

The baseline is local commit 258eef4. Table 1 separates implemented mechanisms from the proposed summary protocol.

Current bounds include 256 CS versions, 16 observed applications per composite, 64 proposals per composition kind, eight indexed alternatives per token, and 32 partners per awakened unit. Dependency traversal has a separate bound. Unknown ownership can cause broad invalidation. Unsupported nested injections, missing pre-update table witnesses, multi-datatype programs, and some scalar interfaces remain explicit limitations.

### 10.2 Bookkeeping cost and its limits

Let  $q$  be changed resources,  $s$  subscription hits, and  $w$  awakened or newly promoted units. After interfaces have been formed, let  $L$  bound interface work,  $K$  candidate partners, and  $B$  the work of a local dependency check. A conditional bookkeeping estimate is

$$O(q \log N_{\text{res}} + s \log N_{\text{unit}} + w(L + KB)), \quad (20)$$

where the two  $N$  values count tracked resources and units. This excludes interface construction, environment checks, serialization, state-isomorphism checks, and the sum of native ripen costs. A dense subscription can still wake many units. Attempt budgets do not bound the worst-case time of one native rule call; the elapsed budget is checked between jobs. These limits prevent a bookkeeping estimate from becoming an unsupported end-to-end complexity claim.

### 10.3 Recursive promotion and locality

A native example contains three CS pairs,  $A \rightarrow A_1 \rightarrow A_2$ ,  $B \rightarrow B_1 \rightarrow B_2$ , and  $C \rightarrow C_1 \rightarrow C_2$ , sharing a boundary parameter. It produces nine closed results, seven shared states, and six promotions, with maximum depth two. Recorded counters include six layer visits, 75 pair checks, 14 resource notifications, and three subscriber wakes [3].

This establishes that closed results can participate in higher-level combinations. It does not establish compression or acceleration. In a separate registry test, union of two resources wakes their two subscribers among 100 independent units. Shared constants, common tables, or unknown mutation ownership can legitimately require more notification work.

### 10.4 Historical records are not a controlled ablation

| Recorded configuration           | Closed results | Shared states |
|----------------------------------|----------------|---------------|
| Math, six rounds, Use only       | 14             | 5             |
| Math, six rounds, ancestor cones | 18             | 7             |
| Math, six rounds, explicit CS    | 16             | 6             |
| Three-CS recursive example       | 9              | 7             |

Table 2: Historical prototype observations. These are not all rerun at one commit; the last row also uses a different input and budget.

The first three records use the same named input and nominal attempt budget, but reflect different implementation versions. They document an evolution rather than isolate a causal improvement. In particular, explicit CS composition did not exceed the earlier cone record in shared-state count.

A separate Python storage prototype expanded 100 C3 triggers into 1,200 Add occurrences, then globally deduplicated them to 304 rows. Retained object bytes including its index and union-find decreased from 25,740 to 20,540, about 20.2%, while queries and joins became slower. This was not egglog RSS and did not replace native tables. It illustrates why sharing potential must be measured together with access and maintenance costs.

For a single snapshot, if  $N(U_j)$  is the e-node set covered by instance  $j$ , coverage must use a union:

$$\text{Coverage} = \frac{|\bigcup_j N(U_j)|}{|N(G)|}. \quad (21)$$

Multiplying instance counts by template sizes overcounts overlap. Net storage must also include dictionaries, bindings, residual facts, indexes, certificates, versions, and any retained raw history. Shared output identities must remain consistent across instances.

## 11 Limitations and Research Agenda

Finite recurrence evidence is not induction. Frequency and compression ratio do not replace guard preservation, a ranking argument, or query completeness. Likewise, recognizing a dissipative family does not prove that other rules cannot restart it.

Closed-content sharing and live instance substitution remain separate. Opaque parameters omit external structure; version tracking prevents unchecked reuse but does not itself validate a current whole-graph embedding. Subsumption and mutable tables require their operational contracts rather than the pure closure identity.

The discovery procedure is intentionally incomplete. Candidate and dependency bounds control work, and its cost depends on changed resources, subscription hits, and bounded pairing rather than on a promise

to find every useful subgraph. Source-environment checks and report construction still have costs. Deeper recursive descriptions can also increase maintenance overhead.

The next evaluation should run all variants at one revision, with matched inputs and budgets, separately ablating Use admission, CSCS, CCSS, recursive promotion, subscriptions, and certified fractal summaries. It should measure repeated internal matching, retained memory, interface-query cost, invalidation/revalidation cost, and omitted candidates, including workloads on which the method loses.

This report uses locally available source and bibliography metadata only. It does not provide an exhaustive related-work survey or establish novelty priority over unreviewed work on graph grammars, library learning, or recursive summaries.

## 12 Conclusion

We describe locality in rule history through explicit interfaces, bindings, and effect obligations. Coarse and smooth applications provide a boundary language; CSCS and CCSS provide distinct composition contracts; ripen and exact state comparison let different processes share closed contents; and event subscriptions and promotion connect these mechanisms into a hierarchy.

Dissipative and infinite-prefix fractal rules offer a parameterized next step. Finite processes can use rankings and fold contracts, whereas unbounded families need lazy positions and a separate symbolic-coverage proof. The implemented prototype supports finite closure composition, recursive promotion, and selected narrow scalar queries. Integrating certified summaries into ripen’s matching interface, and demonstrating reduced native search or storage cost, remain the central hypotheses to test.

## References

- [1] Y. Zhang, Y. R. Wang, O. Flatt, D. Cao, P. Zucker, E. Rosenthal, Z. Tatlock, and M. Willsey. Better Together: Unifying Datalog and Equality Saturation. *Proceedings of the ACM on Programming Languages*, 7(PLDI), Article 125, 2023. DOI: [10.1145/3591239](https://doi.org/10.1145/3591239). Bibliographic metadata and abstract checked against local `egglog/CITATION.bib`; full text not independently reviewed for this report.
- [2] *fractal\_egraph local prototype*, commit 258eef4, 2026. Core mechanisms: `src/native_cs.rs`, `src/closure_contract.rs`, `src/closed_state.rs`; summary schemas: `src/native_bake.rs` and `rules/reduce.egg`. The accompanying verification manifest identifies additional source evidence. Implementation evidence, not an external publication.
- [3] Local experiment records and regression tests, 2026. Records under `experiments/closed_storage/`: `closure-contracts-results.json`, `dependency-candidates-math6.json`, `cs-composition-math6.json`, and `results.json`. Reproduction commands appear in Appendix A; additional test references are listed in the verification manifest. Records are scoped observations, not a uniform benchmark.

## A Reproduction and Evidence Levels

```
# Native finite-property regressions
cargo test --release --test closure_contracts --test cs_composition \
  --test closed_state --test table_support --test closed_pipeline \
  --test layer_rounds --test ripen --test native_history --test native_debug

# Use a fresh output directory
EGG_LAYOUT_RIPEN_JOBS=128 EGG_LAYOUT_RIPEN_PER_BOUNDARY=32 \
EGG_LAYOUT_RIPEN_MILLISECONDS=1000000 \
cargo run --release --analyze --recapture-tier0 \
  --source experiments/closed_storage/recursive-cs.egg \
  --output out/paper-recursive-cs
```

| Evidence                | Supported statement                    | Not implied                      |
|-------------------------|----------------------------------------|----------------------------------|
| Committed event         | A recorded effect occurred             | Every binding can execute it     |
| Finite witness          | A transfer fits observed cases         | Arbitrary-count stability        |
| Local Closed            | Fixed-entry local execution stabilized | Every embedding is closed        |
| Exact state match       | Declared contents can be shared        | Triggers are interchangeable     |
| Ranking proof           | Termination under its supply contract  | Termination under all injections |
| Symbolic query contract | Coverage of specified observations     | No observer needs intermediates  |

Table 3: Evidence levels used throughout the report.